This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Smart Replay: Adaptive Scheduling of Memory Rehearsal for Computational Resource-Aware Incremental Learning

Jianting Chen[1,*] Dianzhi Yu[2] Irwin King[2]
[1]Tongji University [2]The Chinese University of Hong Kong
tj_chenjt@tongji.edu.cn

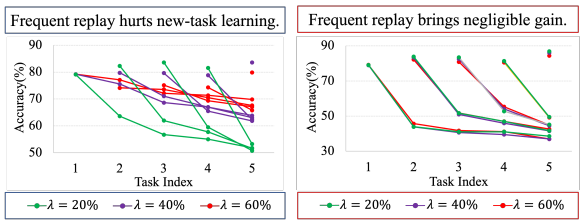
Abstract

Incremental learning (IL) arises from the need to continuously update models under limited data and computational resources. Most existing IL studies focus on data-scarce settings. They often develop complex methods that rely on heavy computation, while overlooking the computational resource constraints common in real-world scenarios. This motivates us to formalize the problem of Computational Resource-Aware Incremental Learning, which explicitly considers the computational budget during model training. To tackle this problem, we propose Smart Replay, an efficient memory rehearsal algorithm that adaptively allocates resources by scheduling the replay ratio across mini-batches. We cast replay-ratio optimization into an optimal control formulation that jointly minimizes new-task and memory losses. We further propose a heuristic Q-function to guide ratio adjustments, adaptively balancing short-term efficiency and long-term stability. Finally, we develop a practical algorithm that periodically updates the replay ratio during training. Experiments on multiple benchmarks validate that Smart Replay consistently outperforms fixed-replay baselines, achieving higher accuracy and lower forgetting under the same computational budget.

1. Introduction

Incremental learning (IL) aims to continuously acquire new knowledge from data streams while preserving prior knowledge, without full retraining on the entire dataset [11, 24]. IL often operates under two constraints: 1) restricted access to past data due to privacy or storage limitations, and 2) limited computational resources for processing large-scale streaming data. These constraints are pervasive in practice.

Existing IL methods [42, 51] can be roughly categorized into regularization-based [21, 52], architecture-based [37, 55, 60] and memory-based approaches [9, 38, 50, 53], which mitigate forgetting by constraining parameter update, modifying network architectures through freezing or expansion, and replaying historical samples. Although effective in preserving prior knowledge, most methods incur significant computational and structural overhead, limiting their deployment in resource- or time-constrained settings where models must be updated daily, or even hourly [14, 49].

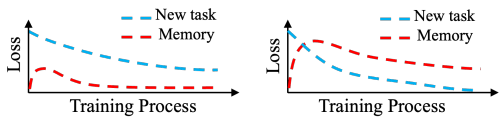


(a) Large Memory Buffer (b) Small Memory Buffer

Figure 1. Illustration showing that more frequent replay is not always beneficial. The accuracy of each task throughout the entire IL process is evaluated on CIFAR-100 under the 5-task Class-IL setting, with replay ratios $\lambda = 20\%, 40\%, 60\%$.

Consequently, training efficiency has become a critical requirement in practical systems. However, existing IL paradigms rarely account for explicit computational constraints. Inspired by recent advances in efficient training techniques [2, 16, 40], we propose the **Computational Resource-Aware Incremental Learning (CRIL)** setting, an IL paradigm designed to operate under limited computational budgets. The key research question is: *When computation is constrained, how can a model adapt in real time while maintaining past knowledge?*

Our work primarily focuses on improving training efficiency, so we relax the constraint on data availability, thereby enabling the use of memory rehearsal. However, replay inevitably consumes a portion of computational resources. Moreover, we find that allocating more resources to replay does not always lead to better performance under limited computational resources. As shown in Fig. 1(a), when the memory buffer is large, frequent replay (large replay ratio) helps preserve prior knowledge, but hinders the learning of new tasks. In contrast, Fig. 1(b) shows that with a smaller buffer, frequent replay yields little benefit. There-



(a) Large Replay Ratio (b) Small Replay Ratio

Figure 2. Illustration showing that a suboptimal replay ratio causes imbalanced convergence under a limited number of epochs.

fore, our objective is to achieve an optimal balance of computational resources between learning new tasks and replaying memory samples within constrained budgets.

The key challenge arises from the mismatch in the learning dynamics of new and memory samples. Memory samples have already been optimized. Subsequent training often causes an initial rise in loss, followed by a rapid decline. In contrast, new task samples require complete optimization from scratch. As a result, their computational demands are heterogeneous and evolve over time. A suboptimal and fixed replay ratio leads to imbalanced convergence, as shown in Fig. 2. To address this challenge, we formulate our objective as a dynamic computational resource allocation task under a fixed budget, where the model adaptively schedules the replay ratio during training.

We propose **Smart Replay**, an efficient memory rehearsal method that adaptively adjusts the ratio of replay samples within each mini-batch. We model each task's training process as a discrete-time dynamic system. Under the optimal control framework, we aim to maximize loss reduction by adjusting the replay ratio in each mini-batch. Within this formulation, a Q-function is defined to estimate the expected benefit of different replay ratios. To handle the challenge of heterogeneous learning dynamics, we design a heuristic approximation of the Q-function that balances short-term efficiency and long-term stability. Guided by the gradients from recent mini-batches, Smart Replay adaptively schedules the replay ratio for upcoming mini-batches.

Experimental results on multiple IL benchmarks demonstrate that Smart Replay consistently improves training efficiency and knowledge retention across various settings. Compared with fixed replay ratios, it achieves higher accuracy and lower forgetting at the same computational cost.

In summary, our main contributions are as follows:

- We propose the CRIL setting, which formulates IL under computational constraints as an optimization problem that explicitly balances efficiency and performance.
- We model the adjustment of the replay ratio within each mini-batch as an optimal control problem. An approximated Q-function is leveraged to enable replay scheduling, thereby achieving dynamic resource allocation.
- Experiments across various memory and resource settings show that Smart Replay achieves higher accuracy and lower forgetting within the same computational budget.

2. Related Work

2.1. Memory-based Incremental Learning

Memory rehearsal stands out for its simplicity and generality among IL methods. Existing methods primarily focus on the construction and utilization of memory buffer.

The construction of memory buffers involves retaining a subset of past samples from the data stream. Beyond random sampling [3, 4, 9], selection strategies based on representativeness [17, 35], diversity [1, 12], and importance [36, 54] are employed to maximize the informativeness of stored samples. Given a fixed buffer capacity, dynamic memory management mechanism is further required to accommodate an increasing number of tasks [6].

The utilization of memory buffers is often coupled with regularization- or architecture-based methods beyond simple rehearsal. Techniques such as knowledge distillation [4, 5, 35, 47], gradient projection [8, 27], dynamic expansion [46, 56] and contrastive learning [3, 45] have been employed to better preserve acquired knowledge. More specifically, memory buffers are typically either mixed with new data for joint sampling [35, 56], or sampled independently at a fixed ratio [8, 27]. However, both strategies have inherent limitations, leaving room for more effective utilization, as explored in this work.

Memory rehearsal incurs substantial computation, especially in online IL [26, 43, 45], where each sample is processed only once to ensure efficiency. Our CRIL setting is similar to online IL but relaxes the strict single-pass constraint. This enables a better balance between performance and efficiency through controlled replay.

2.2. Data-based Efficient Training

Data-based efficient training improves learning efficiency and generalization by optimizing data utilization, primarily through remove and reorder [2].

Remove-based methods focus on pruning redundant samples or constructing representative coresets to reduce computational cost, guided by criteria such as feature distribution [7, 40, 57], learning dynamics [16, 31, 33, 34, 41], gradient contribution [19, 29, 32, 48] and hybrid strategies [10, 28]. These principles have been widely adopted in IL to construct more compact and efficient memory buffers.

Reorder-based methods emphasize dynamic data scheduling, deciding when each sample should participate in training. Curriculum learning [44] provides samples in an easy-to-difficult order, while recent adaptive strategies identify samples based on their losses and prediction confidence [18, 30, 33]. In the IL context, several works have explored replay scheduling at the task or cluster level [22, 39]. We move toward finer-grained control at the batch level, dynamically determining how many memory samples to replay for more efficient IL.

3. Problem Formulation

IL aims to enable model parameters $\boldsymbol{\theta}$ to sequentially learn a task stream $\mathcal{T} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_T\}$, where each dataset $\mathcal{D}_t = \{(x_{t,i}, y_{t,i})\}_{i=1}^{n}$ is drawn from a task-specific distribution $\mathcal{P}_t(x, y)$. In the periodic model-update scenario, all tasks are assumed to contain the same number of samples n. At stage t, the model updates its parameters $\boldsymbol{\theta}_t$ using the loss function $\mathcal{L}$ on the new data $\mathcal{D}_t$, while retaining knowledge acquired from previous tasks $\mathcal{D}_1, \ldots, \mathcal{D}_{t-1}$.

Based on the IL setting, we further define the Computational Resource-Aware IL (CRIL) problem as follows.

Definition 1 (Computational Resource-Aware IL) *Let q represent the average computational cost of processing a single sample. At stage t, the model accesses $N_t \geq n$ samples, with a total sample processing count of $C_t = \sum_{i=1}^{N_t} a_{t,i}$, where $a_{t,i}$ denotes the number of times the i-th sample is used for training. The IL optimization under a fixed computational budget U_t is formulated as:*

$$\begin{aligned} \min_{\boldsymbol{\theta}_t} \quad & \mathbb{E}_{(x,y)\sim\bigcup_{j=1}^{t}\mathcal{P}_j(x,y)}\mathcal{L}(\boldsymbol{\theta}_t; x, y) \\ \textit{s.t.} \quad & q\,C_t \leq U_t. \end{aligned} \tag{1}$$

Memory rehearsal increases the accessible sample size N_t. A memory buffer $\mathcal{M}_t$ stores a subset of samples from previous tasks for replay. Given a memory retention ratio $\delta_j \in (0, 1]$ for each task j, the memory buffer at stage t is defined as $\mathcal{M}_t = \bigcup_{j=1}^{t-1} \text{Subset}(\mathcal{D}_j, \delta_j n)$. Assuming all tasks share a fixed δ, the total number of accessible samples is $N_t = |\mathcal{D}_t| + |\mathcal{M}_t| = n[1 + \delta(t-1)]$.

The computational budget U_t is a controllable factor. Assume that training $\mathcal{D}_t$ alone requires e epochs, e is typically small due to limited computational resources, implying a baseline budget $U_{\text{base}} = q\,e\,n$. We introduce a budget growth coefficient $\vartheta \in [0, 1]$ that controls how the total training budget scales over time: $U_t = U_{\text{base}}[1 + \vartheta(t-1)]$. When $\vartheta = 0$, the budget remains fixed across tasks, whereas larger ϑ values allocate proportionally more computational resources as IL progresses.

When δ = 1 and ϑ= 1, the process is equivalent to fully retraining all data, achieving optimal performance. In practice, however, the growth of the computational budget usually lags behind the expansion of the accessible sample size, highlighting the necessity of efficient resource allocation.

4. Proposed Method

In this section, we begin with the motivation and objective (Sec. 4.1), followed by an optimal control formulation of replay ratio scheduling (Sec. 4.2). We then derive a heuristic Q-function approximation to guide adaptive control (Sec. 4.3), and develop an efficient algorithm for practical implementation (Sec. 4.4).

4.1. Motivation and Objective

We study how to allocate a limited computational budget across all accessible samples. For a given ϑ, the total number of processed samples at stage t is fixed as $C \leq U/q$ (for simplicity, the stage index t is omitted). Following the mini-batch gradient descent paradigm, the total processed samples are divided into $R = C/b$ mini-batches, each producing one parameter update and yielding the parameter sequence $\boldsymbol{\theta}_0, \boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_r, \ldots, \boldsymbol{\theta}_R$. Each mini-batch B_r consists of b samples and is composed of both memory samples and new-task samples: $B_r = B_r^M \cup B_r^D$. We first define $\lambda \in [0, 1]$ as a fixed replay ratio shared across all mini-batches, i.e., $|B_r^M| = \lambda b$. To enable adaptive scheduling, we further introduce a dynamic replay ratio $\lambda_r \in [0, 1]$ for the r-th mini-batch. By dynamically optimizing λ_r, the sizes of B_r^D and B_r^M can be adaptively controlled, enabling computational resources to be allocated at the micro level and thereby improving overall efficiency at the macro level.

Memory and new-task samples exhibit distinct learning dynamics. An inappropriate replay ratio may cause convergence imbalance: insufficient replay causes forgetting of previously learned knowledge, whereas excessive replay results in overfitting to old tasks and underfitting to new ones. Moreover, in the early phase of training, when new-task learning dominates, a lower replay ratio is preferred to enhance model plasticity. As the new task gradually converges, a higher replay ratio becomes beneficial for preserving prior knowledge. Therefore, the replay ratio should adapt to the model's training state.

Building upon the above motivation, we further formalize the objective. Given a memory buffer $\mathcal{M}$ and a new-task dataset $\mathcal{D}$, the losses on the memory and new tasks after R mini-batch updates are denoted as l_R^M and l_R^D, respectively. The model aims to jointly minimize these two losses, which is equivalent to maximizing the total loss reduction:

$$\min(l_R^M + l_R^D) \quad \Leftrightarrow \quad \max \sum_{r=0}^{R-1} \left(\Delta l_r^M + \Delta l_r^D\right), \tag{2}$$

$$\Delta l_r^M = l_r^M - l_{r+1}^M, \qquad \Delta l_r^D = l_r^D - l_{r+1}^D. \tag{3}$$

Here, Δl_r^M and Δl_r^D represent the loss reduction contributed by the r-th update with mini-batch B_r. Since the composition of B_r depends on the replay ratio λ_r, both loss reductions are functions of λ_r, expressed as $\Delta l_r^M = \Delta\mathcal{L}_r^M(\lambda_r)$ and $\Delta l_r^D = \Delta\mathcal{L}_r^D(\lambda_r)$. Therefore, the optimization objective with respect to the replay ratio can be formulated as:

$$\max_{\{\lambda_r\}_{r=0}^{R-1}} \sum_{r=0}^{R-1} \left(\Delta\mathcal{L}_r^M(\lambda_r) + \Delta\mathcal{L}_r^D(\lambda_r)\right). \tag{4}$$

4.2. Optimal Control Formulation

Optimal control (OC) provides a principled framework for sequential decision problems [20], enabling the formulation of optimization objectives over dynamic processes. The replay ratio scheduling problem can thus be naturally formulated within this framework.

Specifically, we model the training process as a discrete-time dynamical system at iteration steps $r = 0, 1, \ldots, R-1$. At step r, the system state is denoted by the tuple $(l_r^M,\, l_r^D,\, r)$ and evolves according to the transition rules in Eq. (3). The control variable at each step, $\lambda_r \in [0, 1]$, determines the proportion of memory and new-task samples in each mini-batch. The objective is to determine the sequence $\{\lambda_r\}_{r=0}^{R-1}$ that maximizes the cumulative loss reduction outlined in Eq. (4).

Following the OC framework, the immediate stage reward at step r is defined as:

$$R(l_r^M, l_r^D; \lambda_r) = \Delta l_r^M + \Delta l_r^D, \tag{5}$$

which measures the joint loss reduction. The corresponding cumulative reward is represented by the value function:

$$V(l_r^M, l_r^D, r) = \max_{\lambda_{r:R-1}} \sum_{j=r}^{R-1} R(l_j^M, l_j^D; \lambda_j), \tag{6}$$

which denotes the maximum total loss reduction attainable from the current state (l_r^M, l_r^D, r) until the end of training.

According to the Bellman equation, the value function satisfies the recursive relation:

$$V(l_r^M, l_r^D, r) = \max_{\lambda_r} \left[R(l_r^M, l_r^D; \lambda_r) + V(l_{r+1}^M, l_{r+1}^D, r+1)\right], \tag{7}$$

where the term inside the brackets can be regarded as the state-action value function $Q(l_r^M, l_r^D, r; \lambda_r)$. No discount factor is included, as this is a finite-horizon optimization where all steps are equally weighted. The optimal control at step r is determined by:

$$\lambda_r^* = \operatorname*{argmax}_{\lambda_r} Q(l_r^M, l_r^D, r; \lambda_r). \tag{8}$$

This formulation yields an adaptive control policy for λ_r, achieving temporally optimal scheduling.

4.3. Heuristic Q-Function Approximation.

The Q-function follows the dynamic programming structure and can be divided into two components: 1) a stage reward function that measures the immediate loss reduction, and 2) a value function that represents the potential future gain.

While the stage reward can be simply defined as the sum of loss reductions, we further add a smoothness regularization term to prevent fluctuations between adjacent steps:

$$R(l_r^M, l_r^D; \lambda_r) = \Delta l_r^M + \Delta l_r^D - \epsilon(\lambda_r - \lambda_{r-1})^2. \tag{9}$$

Since the value function reflects the maximum total loss reduction achievable from the current step to the end of training, it is challenging to quantify precisely. Considering that losses typically exhibit an exponentially decaying pattern, we heuristically design:

$$V(l_r^M, l_r^D, r) = \left(l_r^M - l_r^M e^{\rho_M \frac{r-R}{R}}\right) + \left(l_r^D - l_r^D e^{\rho_D \frac{r-R}{R}}\right), \tag{10}$$

where $l_r^M e^{\rho_M (r-R)/R}$ and $l_r^D e^{\rho_D (r-R)/R}$ represent the predicted losses at the end of training, and $\rho_M > 0$ and $\rho_D > 0$ denote the decay rates of the memory and new-task losses, respectively. A larger decay rate indicates faster convergence. Combining the two parts, the Q-function can be written as:

$$Q(l_r^M, l_r^D, r; \lambda_r) = w_r^M \Delta l_r^M + w_r^D \Delta l_r^D - \epsilon(\lambda_r - \lambda_{r-1})^2, \tag{11}$$

$$w_r^M = e^{\rho_M (r-R)/R}, \qquad w_r^D = e^{\rho_D (r-R)/R}. \tag{12}$$

Here, w_r^M and w_r^D are time-dependent exponential weights for the two tasks, and terms independent of λ_r are omitted.

To further quantify Δl_r^M and Δl_r^D, we linearize the training dynamics around $\boldsymbol{\theta}_r$ using a first-order Taylor expansion:

$$\Delta l_r^M \approx \nabla L_r^M \, \Delta \boldsymbol{\theta}_r, \quad \Delta l_r^D \approx \nabla L_r^D \, \Delta \boldsymbol{\theta}_r, \tag{13}$$

where ∇L_r^M and ∇L_r^D are task gradients. The parameter update, given a learning rate η, is expressed as:

$$\Delta \boldsymbol{\theta}_r = \eta \left[\lambda_r \nabla L_r^M + (1 - \lambda_r) \nabla L_r^D\right]. \tag{14}$$

Substituting into the Q-function yields:

$$\begin{aligned} &Q(l_r^M, l_r^D, r; \lambda_r) \\ &= \eta w_r^M \lambda_r \|\nabla L_r^M\|^2 + \eta w_r^D (1 - \lambda_r) \|\nabla L_r^D\|^2 \\ &+ \eta \big(w_r^M (1 - \lambda_r) + w_r^D \lambda_r\big) \nabla L_r^M \cdot \nabla L_r^D \\ &- \epsilon(\lambda_r - \lambda_{r-1})^2. \end{aligned} \tag{15}$$

Let $\gamma_r = w_r^D \|\nabla L_r^D\|^2$, and denote β as the angle between ∇L_r^M and ∇L_r^D. The Q-function can be reformulated as (omitting terms independent of λ_r):

$$\begin{aligned} &Q(l_r^M, l_r^D, r; \lambda_r) \\ &= \eta \gamma_r \lambda_r \left[\tfrac{w_r^M}{w_r^D} \tfrac{\|\nabla L_r^M\|^2}{\|\nabla L_r^D\|^2} + (1 - \tfrac{w_r^M}{w_r^D}) \tfrac{\|\nabla L_r^M\|}{\|\nabla L_r^D\|} \cos\beta - 1\right] \\ &- \epsilon(\lambda_r - \lambda_{r-1})^2. \end{aligned} \tag{16}$$

This expression is quadratic in nature, making it straightforward to solve and contains four key factors:

1) the **long-term weight ratio** w_r^M / w_r^D, which captures the time-adaptive weighting between tasks;

2) the **short-term gradient norms** $\|\nabla L_r^M\|$ and $\|\nabla L_r^D\|$, which determine the immediate contributions of individual tasks;
3) the **gradient-direction correlation** $\cos\beta$, which reflects the synergy or conflict between tasks;
4) the **smoothness regularization** term, which ensures stable control.

These factors enable λ_r to perform time-adaptive scheduling. The effectiveness of these factors is further examined via ablation studies in Sec. 5.4. The detailed derivations from Eq.(9) to Eq.(16) are shown in Appendix F.

4.4. Periodic Adaptive Scheduling

It is infeasible to compute the full loss and gradients ∇L_r^M and ∇L_r^D after every parameter update. In practice, only the gradients obtained from the current mini-batch are available. To obtain robust gradient estimates under this constraint, we adopt a moving-average approximation scheme.

Specifically, we update λ_r once every z steps. Within each interval, we compute the estimated gradient norm as:

$$\widehat{\|\nabla L_r^\Omega\|} = \frac{1}{z}\sum_{j=r-z}^{r-1} \|\nabla_\theta L(\boldsymbol{\theta}_j; B_j^\Omega)\|, \Omega \in \{M, D\}, \quad (17)$$

where $\Omega \in \{M, D\}$ indicates the data source, either memory M or new-task D. Here, $\nabla_\theta L(\boldsymbol{\theta}_j; B_j^\Omega)$ represents the gradient of the loss computed on the corresponding mini-batch. Note that r is the current update step, where $r \in \{z, 2z, 3z, \dots\}$. The variable $j \in [r-z,\, r-1]$ indexes the preceding steps within the corresponding interval.

Similarly, the cosine of the angle between the gradient directions is approximated as:

$$\widehat{\cos\beta} = \frac{1}{z}\sum_{j=r-z}^{r-1} \frac{\langle \nabla_\theta L(\boldsymbol{\theta}_j; B_j^M), \nabla_\theta L(\boldsymbol{\theta}_j; B_j^D)\rangle}{\|\nabla_\theta L(\boldsymbol{\theta}_j; B_j^M)\|\, \|\nabla_\theta L(\boldsymbol{\theta}_j; B_j^D)\|}. \quad (18)$$

Building upon the above estimations, we maximize the Q-function with respect to λ_r to obtain a closed-form solution:

$$\lambda_r = \lambda_{r-1} + \eta\frac{\gamma_r}{2\epsilon}\left[\frac{w_r^M}{w_r^D}\frac{\|\widehat{\nabla L_r^M}\|^2}{\|\widehat{\nabla L_r^D}\|^2} + \left(1-\frac{w_r^M}{w_r^D}\right)\frac{\|\widehat{\nabla L_r^M}\|}{\|\widehat{\nabla L_r^D}\|}\widehat{\cos\beta} - 1\right], \quad (19)$$

$$\frac{w_r^M}{w_r^D} = e^{(\rho_M-\rho_D)(r-R)/R} = e^{\Delta\rho(r-R)/R}. \quad (20)$$

We define $\gamma = \gamma_r/(2\epsilon)$ and $\Delta\rho = \rho_M - \rho_D$, which are treated as hyperparameters. Here, γ controls the update step size of λ_r, and $\Delta\rho$ characterizes the difference of decay rates between memory and new tasks. A positive and larger $\Delta\rho$ implies that memory samples converge more easily, prompting the model to reduce the replay ratio in early training and thereby enhance its adaptation to new tasks. Note that $\Delta\rho$ is related to the memory retention ratio δ. A smaller δ (i.e., a smaller memory buffer) leads to faster fitting of memory samples, and thus requires a larger $\Delta\rho$. The hyperparameter analysis in Sec. 5.5 supports this claim.

For initialization, we simply set $\lambda_0 = \tau$, where τ is a hyper-parameter. In the experiments, sensitivity analyses are conducted on all hyperparameters $\{z, \gamma, \Delta\rho, \tau\}$. Fig. 3 illustrates the periodic adaptive scheduling process, and the corresponding pseudo-code is shown in Algorithm 1.

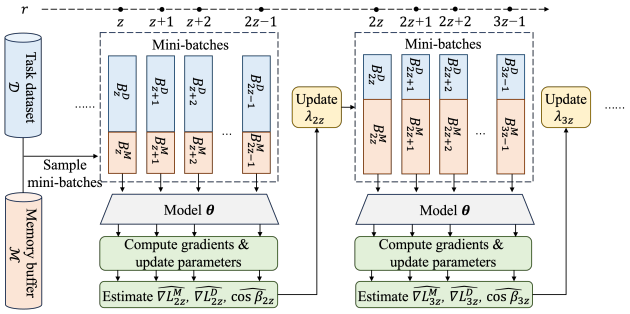


Figure 3. Illustration of periodic adaptive scheduling

Algorithm 1: Periodic Adaptive Scheduling

Input: Memory buffer $\mathcal{M}$, task dataset $\mathcal{D}$, interval length z, hyperparameters $\{\gamma,\, \Delta\rho,\, \tau\}$
Output: Adaptive replay ratio $[\lambda_r]_{r=0}^{R-1}$
Set $\lambda_0 = \tau$;
for *each step* $r = 0, 1, \dots, R-1$ **do**
  if $r \mod z = 0$ *and* $r > 0$ **then**
    Estimate $\widehat{\|\nabla L_r^M\|}$, $\widehat{\|\nabla L_r^D\|}$, and $\widehat{\cos\beta}$ using Eqs. (17)–(18);
    Compute w_r^M/w_r^D from Eq. (20);
    Update λ_r by the solution in Eq. (19);
  Sample $B_r^M \sim \mathcal{M}$ and $B_r^D \sim \mathcal{D}$ according to λ_r;
  Form mini-batch $B_r = B_r^M \cup B_r^D$;
  Compute gradients and update parameters;

5. Experiments

To evaluate Smart Replay, we conduct experiments to answer the following research questions.

RQ1. How effective is Smart Replay under different memory capacities and computational budgets?
RQ2. How does adaptive scheduling adjust the replay ratio and affect performance?
RQ3. What is the contribution of each key factor in the Q-function to the overall effectiveness?
RQ4. How sensitive is Smart Replay to hyperparameters?

Table 1. Average accuracies (%) under the limited-memory setting with different replay ratios (λ) and resource budgets (ϑ)

Replay ratio	CIFAR-100 ϑ=0	CIFAR-100 ϑ=0.1	CIFAR-100 ϑ=0.2	Tiny-ImageNet ϑ=0	Tiny-ImageNet ϑ=0.1	Tiny-ImageNet ϑ=0.2	Rotated-MNIST ϑ=0	Rotated-MNIST ϑ=0.1	Rotated-MNIST ϑ=0.2
ER w/ limited memory									
Union	62.30	62.72	62.22	41.40	42.87	43.60	88.32	88.98	89.45
λ=0.1	59.25	61.44	62.37	33.82	38.07	40.61	86.13	87.16	87.95
λ=0.2	61.94	62.41	62.60	41.06	43.22	43.96	88.50	89.18	89.67
λ=0.3	61.89	61.45	62.17	42.78	42.79	42.71	89.37	89.93	90.28
λ=0.4	61.53	61.75	61.87	41.63	41.53	41.20	89.73	90.13	90.61
$\lambda \nearrow$	63.84	63.57	63.20	**43.09**	43.89	44.54	89.66	90.16	90.54
$\lambda \searrow$	57.91	58.30	58.17	33.50	33.94	33.82	88.91	89.45	89.79
Smart	**64.58**	**63.88**	**63.83**	42.37	**46.51**	**47.70**	**89.87**	**90.43**	**90.77**
iCaRL w/ limited memory									
Union	62.25	62.65	62.61	42.46	42.68	42.58	89.17	89.69	90.10
λ=0.1	61.74	62.41	62.56	37.38	40.73	41.93	88.71	89.43	89.94
λ=0.2	62.90	63.40	62.81	41.93	42.68	43.41	89.63	90.21	90.67
λ=0.3	63.38	63.47	63.33	42.10	42.47	42.25	89.98	**90.53**	90.89
λ=0.4	63.35	63.67	63.63	42.01	41.50	41.24	**90.03**	90.47	**90.93**
$\lambda \nearrow$	65.16	**65.87**	**65.95**	44.75	44.80	44.50	88.00	88.55	89.95
$\lambda \searrow$	60.28	61.39	61.55	33.37	34.60	34.67	89.65	90.23	90.00
Smart	**65.43**	65.49	65.88	**46.60**	**49.08**	**48.55**	89.87	90.31	90.69
MEMO w/ limited memory									
Union	63.37	63.89	**64.34**	43.78	45.99	46.92	79.94	81.69	82.82
λ=0.1	62.08	63.93	63.90	35.55	38.97	41.25	74.92	77.14	78.24
λ=0.2	64.03	64.24	64.46	42.40	44.73	42.13	78.93	81.19	82.56
λ=0.3	64.04	64.25	64.17	44.22	45.70	46.17	81.64	83.31	84.18
λ=0.4	63.40	63.23	65.99	44.24	45.04	44.82	82.35	83.59	84.22
$\lambda \nearrow$	63.38	63.26	62.76	**46.08**	47.17	47.77	82.26	**84.83**	84.72
$\lambda \searrow$	59.96	60.40	60.58	35.40	36.11	36.99	78.98	80.28	81.45
Smart	**65.57**	64.43	64.01	44.50	**47.60**	**49.06**	**82.63**	84.43	**85.33**
STAR w/ limited memory									
Union	62.70	63.81	64.27	42.13	43.47	44.09	91.32	**91.87**	**92.30**
λ=0.1	58.18	60.44	62.15	31.38	34.94	37.22	89.70	90.47	90.96
λ=0.2	64.01	64.47	65.11	37.98	41.33	43.53	90.99	91.59	92.05
λ=0.3	63.77	64.61	65.35	42.44	44.22	44.00	90.89	91.51	91.90
λ=0.4	63.47	63.92	64.50	42.61	43.27	43.65	90.94	91.58	92.03
$\lambda \nearrow$	**65.25**	65.73	66.25	**45.71**	**46.61**	47.67	91.13	91.72	92.15
$\lambda \searrow$	60.71	61.09	62.60	33.71	34.90	35.11	90.73	91.51	91.88
Smart	65.14	**66.77**	**67.27**	45.12	46.14	**47.86**	**91.43**	91.71	92.03

Bold values indicate the best results, and underlined values indicate the second-best results.

5.1. Experimental Setup

Datasets. We evaluate Smart Replay on both Class-IL and Domain-IL benchmarks. For Class-IL, CIFAR-100 [23] and Tiny-ImageNet [25] are used, where 100 and 200 classes are evenly divided into 5 and 10 incremental tasks, respectively. For Domain-IL, Rotated-MNIST [27] is created by evenly splitting the dataset into five subsets and rotating the digits by 0°, 30°, 60°, 90°, and 120°, forming five domain tasks.

Evaluation protocol. We follow the CRIL setting and consider different levels of data availability and computational resources. Two memory settings are designed: limited and unlimited. In the **limited-memory** setting, $\delta = 0.1$, where 10% of each task's data are randomly sampled. In the **unlimited-memory** setting, $\delta = 1$, meaning that all prior data are accessible. For computation, we define three budget levels with $\vartheta = 0, 0.1$, and 0.2. Under the same budget, higher performance indicates greater efficiency.

Baselines. We compare Smart Replay with three types of baselines: i) Union sampling, which draws mini-batches from the union of new data and memory without explicitly controlling the replay ratio; ii) fixed-ratio sampling, where the replay ratio λ is set to different constants. iii) linear heuristics, which adjust λ linearly, either increasing it from 0.2 to 0.8 ($\lambda \nearrow$) or decreasing it from 0.8 to 0.2 ($\lambda \searrow$).

For generality, we further validate our algorithm on several IL methods, including experience replay (ER), the distillation-based iCaRL [35], the lightweight expansion-based MEMO [58], and the weight perturbation-based STAR [13]. Strongly expansion-based frameworks such as DER [46] and TagFex [56] are excluded due to their high computational overheads, which are incompatible with our limited-resource setting.

Evaluation Metrics. To reflect performance throughout the IL process, the *average accuracy* is defined as $A = \frac{1}{T}\sum_{i=1}^{T}\frac{1}{i}\sum_{j=1}^{i}A_{i,j}$, where T denotes the total number of tasks (stages), and $A_{i,j}$ is the test accuracy on task j after learning task i. We also employ the *last forgetting* to quantify the degradation on previous tasks, defined as $F = \frac{1}{T-1}\sum_{i=1}^{T-1}\left(A_{i,i} - A_{T,i}\right)$, and report the results in Appendix B. We present the training time cost in Appendix C, as it remains generally consistent for the same method under the same budget.

Training Details. We adopt different backbone networks according to the dataset difficulty. ResNet-18 [15] is used for CIFAR-100, while ResNet-34 [15] is employed for Tiny-ImageNet. For Rotated-MNIST, a three-layer MLP is used to mimic a scenario with severe forgetting. We only use the gradients of the parameters in the top layer of the network to guide the update of λ_r, improving efficiency.

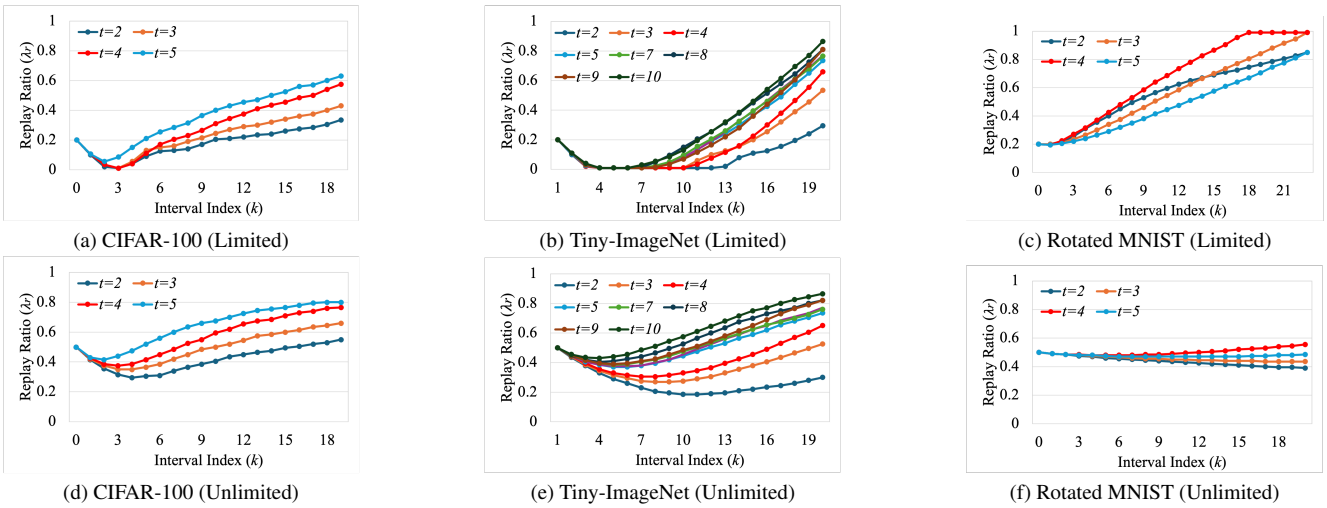
The base budget U_{base} is set to 20 epochs for each task ($e = 20$). *Note that the small number of epochs is intended to emulate a constrained computational environment.* We also provide the results with 60 and 100 epochs in Appendix D. All models are optimized using SGD with a batch size of 200. The initial learning rate is set to 0.1 for CIFAR-100 and Tiny-ImageNet, and 0.01 for Rotated-MNIST, following a cosine annealing schedule.

All experiments are conducted on an NVIDIA RTX 4090 GPU using PyTorch 2.2 and Transformers 4.44. For the main experiments, each setting is repeated three times with different seeds and task orders, and the mean results are reported. More details are provided in Appendix A.

Hyperparameters. We fix $z = 50$ and $\gamma = 1$ for all settings. For the limited-memory setting, $\tau = 0.2$ and $\Delta\rho = 2.0$. For the unlimited-memory setting, we set $\tau = 0.5$ and $\Delta\rho = 0.5$. Details of the sensitivity analysis and parameter ranges are provided in Section 5.5.

5.2. Main Results (RQ1)

The results in the limited-memory and unlimited-memory settings are reported in Table 1 and Table 2, respectively. In the limited-memory setting, Smart Replay exhibits a clear advantage on CIFAR-100. When combined with either the base model or iCaRL, it improves accuracy by



(a) CIFAR-100 (Limited) (b) Tiny-ImageNet (Limited) (c) Rotated MNIST (Limited)

(d) CIFAR-100 (Unlimited) (e) Tiny-ImageNet (Unlimited) (f) Rotated MNIST (Unlimited)

Figure 4. Dynamic changes of replay ratio (λ_r) with adaptive scheduling under two memory settings

Table 2. Average accuracies (%) under the unlimited-memory setting with different replay ratios (λ) and resource budgets (ϑ)

Replay	CIFAR-100			Tiny-ImageNet			Rotated-MNIST		
ratio	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2
ER w/ unlimited memory									
Union	74.16	74.46	75.29	**55.11**	56.83	59.53	**91.55**	91.73	92.22
λ=0.2	68.42	70.27	71.64	43.26	47.09	49.38	88.61	89.64	90.43
λ=0.4	73.20	74.36	75.37	50.84	54.41	56.50	90.56	91.32	91.87
λ=0.6	73.67	75.01	75.87	53.33	57.08	58.96	91.13	91.75	92.20
λ=0.8	72.74	73.91	74.53	53.73	56.91	58.97	90.56	91.35	91.87
$\lambda \nearrow$	72.94	72.96	72.88	52.41	55.56	57.70	90.77	91.41	92.04
$\lambda \searrow$	69.61	69.60	69.75	46.28	50.53	53.27	90.04	91.03	91.61
Smart	**74.70**	**75.43**	**76.30**	53.94	**57.20**	**59.93**	91.47	**92.19**	**92.74**
iCaRL w/ unlimited memory									
Union	**75.25**	74.44	74.72	**54.80**	57.49	**59.91**	89.77	90.55	91.08
λ=0.2	69.32	70.61	71.75	44.86	48.77	51.20	89.62	90.37	90.89
λ=0.4	72.36	73.34	74.26	50.98	54.43	56.35	90.22	90.84	91.45
λ=0.6	73.46	74.25	74.77	53.55	56.83	58.90	90.02	90.59	91.15
λ=0.8	71.65	73.05	73.94	53.52	56.41	58.17	88.65	89.48	90.15
$\lambda \nearrow$	74.05	73.68	73.95	53.25	56.41	58.26	89.23	90.00	90.72
$\lambda \searrow$	71.40	69.84	71.03	44.79	48.95	51.85	90.11	90.81	91.40
Smart	73.82	**74.86**	**75.10**	54.53	**58.03**	59.55	**90.77**	**91.45**	**92.02**
MEMO w/ unlimited memory									
Union	**74.97**	74.55	75.00	**53.34**	55.30	57.73	80.53	82.30	83.30
λ=0.2	69.83	71.22	72.63	43.66	47.36	49.99	76.67	78.67	80.27
λ=0.4	72.78	74.00	74.81	50.35	53.02	55.62	80.77	81.97	83.62
λ=0.6	73.68	69.83	74.98	52.23	54.85	56.83	81.04	**82.47**	83.90
λ=0.8	71.96	73.11	73.70	51.05	54.70	57.02	77.85	79.57	81.20
$\lambda \nearrow$	72.51	73.17	74.54	51.47	54.21	56.29	**81.99**	82.19	83.89
$\lambda \searrow$	72.69	71.50	72.51	43.46	47.90	51.11	75.73	78.10	79.65
Smart	74.30	**74.91**	**75.21**	52.78	**55.84**	**58.17**	81.13	82.35	**84.51**
STAR w/ unlimited memory									
Union	74.54	75.20	76.14	**54.65**	57.22	58.16	91.10	91.79	**92.33**
λ=0.2	67.61	70.49	71.29	37.06	42.43	45.61	89.08	90.04	90.65
λ=0.4	72.69	74.03	75.32	48.69	52.51	55.42	90.69	91.44	92.02
λ=0.6	74.44	75.42	76.25	53.14	56.52	58.01	91.19	**91.85**	92.03
λ=0.8	72.65	73.96	74.62	53.75	56.60	58.61	90.62	91.43	91.99
$\lambda \nearrow$	73.15	73.95	74.54	50.22	53.42	55.96	90.87	91.61	92.15
$\lambda \searrow$	70.30	71.65	72.31	45.23	49.61	52.29	90.36	91.34	91.80
Smart	**74.97**	**75.91**	**76.50**	53.92	**57.81**	**58.66**	**91.38**	91.61	92.20

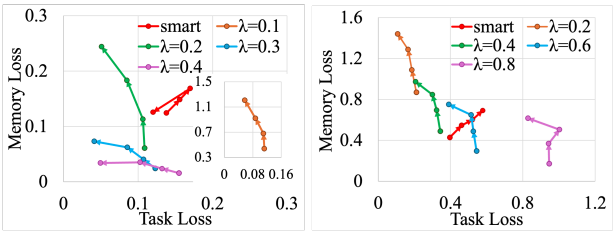
about 2% compared with the best result using a fixed λ. On Tiny-ImageNet, the improvements are even more pronounced, particularly when $\vartheta = 0.1$ or 0.2 across the three IL methods. Owing to limited training resources, the fixed-λ variants perform poorly, whereas Smart Replay achieves nearly 5% higher accuracy by effectively reusing those limited resources. For Rotated-MNIST, a relatively simple task, Smart Replay brings moderate gains for the base model, while the improvement for iCaRL is less prominent because its strong distillation constraint already maintains high performance across replay ratios. Since MEMO is not tailored for domain-IL, its inferior performance is consistent with previous findings [59].

When memory is unlimited, Smart Replay consistently outperforms the fixed-ratio baselines across all datasets and IL methods. As all samples are accessible, sample diversity is significantly enhanced, leading to overall better performance in this setting. Smart Replay delivers stable improvements on CIFAR-100 and Tiny-ImageNet, and achieves slight yet consistent gains on Rotated-MNIST under all three IL methods. Overall, Smart Replay demonstrates strong adaptability to varying resource conditions.

5.3. Evaluation of Adaptive Scheduling (RQ2)

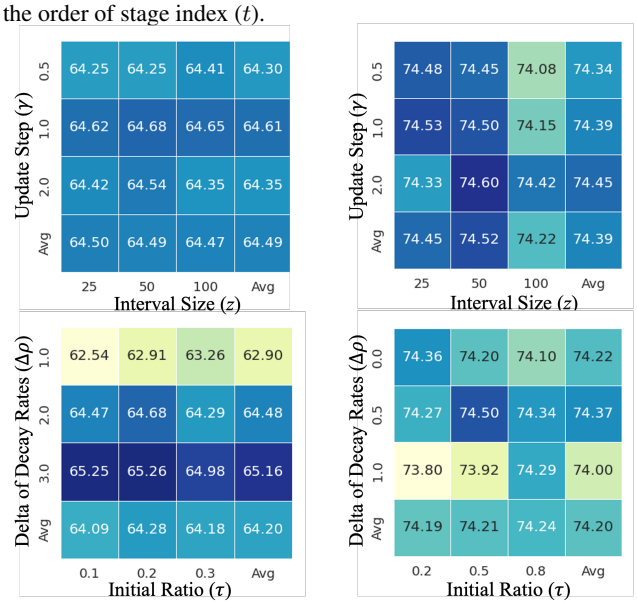
Fig. 4 illustrates the scheduling of the replay ratio λ_r under the minimal computational budget on the base model. Similar configurations are adopted in the following experiments. On CIFAR-100 and Tiny-ImageNet, λ_r exhibits a *weak-to-strong* transition as training progresses: a smaller λ_r at the beginning enhances plasticity for new tasks, while a larger λ_r in later stages restores stability. Early in training, λ_r is guided by the long-term weight ratio of $\Delta\rho$, while later it is driven by gradients to accelerate memory loss reduction. As stage t progresses, the growing number of samples in the memory buffer requires larger replay ratios. A similar but less pronounced trend appears on Rotated-MNIST due to its small learning rate of 0.01.

Fig. 5 further presents the task and memory losses on



(a) Limited Memory (b) Unlimited Memory

Figure 5. Task loss and memory loss after each training stage under different replay ratios (λ) on CIFAR-100. The arrow indicates the order of stage index (t).



(a) Limited Memory (b) Unlimited Memory

Figure 6. Hyperparameter analysis on CIFAR-100

CIFAR-100 after each stage t, corresponding to the objective defined in Eq. (4). With a fixed λ, the task loss decreases while the memory loss increases as t grows, indicating an increasing tendency to forget previous knowledge. In contrast, Smart Replay maintains a better balance between the two losses, with results close to the diagonal in Fig. 5, enabling both the task and memory losses to converge to comparable levels. In the limited-memory setting, although $\lambda = 0.3$ and 0.4 yield smaller training losses on the few memory samples, this phenomenon reflects overfitting, since no clear advantage is observed on the test set. The effect of Smart Replay on the learning dynamics within each stage is presented in Appendix E.

5.4. Ablation Study (RQ3)

We construct four ablation variants to examine the factors in the Q-function (Eq. (16)): *w/o Long* sets $w_r^M/w_r^D = 1$ during training; *w/o Short* keeps $\|\nabla L_r^M\|/\|\nabla L_r^D\| = 1$; *w/o Cosine* fixes $\cos\beta = 0$; and *w/o Smooth* removes the regularization term, degenerating the Q-function into a linear form where λ_r sharply switches between 0 and 1.

Table 3. Results of ablation study

	Limited Memory		Unlimited Memory	
	CIFAR	Tiny	CIFAR	Tiny.
Smart	64.58	42.37	74.70	53.94
w/o Long	61.43	41.39	74.20	52.93
w/o Short	40.20	18.32	56.55	23.54
w/o Cosine	64.03	42.02	74.27	53.23
w/o Smooth	36.89	16.23	39.12	20.12

As shown in Table 3, *w/o Smooth* causes oscillations of λ_r, leading to instability and the worst performance. *w/o Short* loses adaptive control, driving λ_r gradually toward zero and forgetting prior knowledge. The long-term factor plays a crucial role under limited-memory settings, preventing overfitting on memory samples. Other variants exhibit moderate degradation, confirming that all factors contribute to the overall optimization performance.

5.5. Hyperparameter Sensitivity Analysis (RQ4)

We categorize the hyperparameters into two groups. The update step size γ and interval size z determine the step and frequency of λ_r updates, while the initial ratio τ and the decay-rate difference $\Delta\rho$ control the trajectory of λ_r.

Fig. 6 reports the average accuracies on CIFAR-100 under different hyperparameter combinations. In the limited-memory setting, $\Delta\rho$ has a significant impact: increasing it from 1.0 to 3.0 consistently improves performance. A similar pattern is observed in the unlimited-memory setting, where $\Delta\rho = 0.5$ performs better than 0 or 1.0. These observations suggest that $\Delta\rho$, serving as a weighting factor for long-term benefits, is crucial and closely associated with the memory size. However, selecting an appropriate value of $\Delta\rho$ still relies on empirical tuning. In contrast, our method remains robust to variations in the other parameters within a reasonable range.

6. Conclusion and Future Work

This study addresses the efficiency problem in IL, focusing on optimizing computational resource allocation under memory rehearsal. We argue that the replay ratio should follow a *weak-to-strong* principle to balance plasticity and stability. Accordingly, we propose Smart Replay, which employs a heuristic Q-function to adaptively adjust the replay ratio within each mini-batch. Experimental results demonstrate that Smart Replay is efficient and robust across diverse resource conditions.

Limitation and future work. In the current implementation, a fixed set of hyperparameters is used across all IL stages. As the memory buffer accumulates and the convergence dynamics change, adaptively tuning these hyperparameters could further enhance efficiency.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (U23B2057) and was conducted by Jianting Chen during his visit to CUHK, under the guidance of Professor Irwin King.

References

[1] Jihwan Bang, Heesu Kim, Youngjoon Yoo, Jung-Woo Ha, and Jonghyun Choi. Rainbow memory: Continual learning with a memory of diverse samples. In *Proc. of CVPR*, pages 8218–8227, 2021. 2

[2] Brian R. Bartoldson, Bhavya Kailkhura, and Davis W. Blalock. Compute-efficient deep learning: Algorithmic trends and opportunities. *J. Mach. Learn. Res.*, 24:122:1–122:77, 2023. 1, 2

[3] Prashant Shivaram Bhat, Bharath Chennamkulam Renjith, Elahe Arani, and Bahram Zonooz. IMEX-Reg: Implicit-Explicit Regularization in the Function Space for Continual Learning. *TMLR*, 2024. 2

[4] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: A strong, simple baseline. *Proc. of NeurIPS*, 33:15920–15930, 2020. 2

[5] Yuliang Cai, Jesse Thomason, and Mohammad Rostami. Task-Attentive Transformer Architecture for Continual Learning of Vision-and-Language Tasks Using Knowledge Distillation. In *Proc. of EMNLP Findings*, pages 6986–7000, 2023. 2

[6] Francisco M. Castro, Manuel J. Marín-Jiménez, Nicolás Guil, Cordelia Schmid, and Karteek Alahari. End-to-end incremental learning. In *Proc. of ECCV*, pages 241–257, 2018. 2

[7] Chengliang Chai, Jiayi Wang, Nan Tang, Ye Yuan, Jiabin Liu, Yuhao Deng, and Guoren Wang. Efficient coreset selection with cluster-based methods. In *Proc. of KDD*, pages 167–178, 2023. 2

[8] Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with A-GEM. In *Proc. of ICLR*, 2019. 2

[9] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc'Aurelio Ranzato. On Tiny Episodic Memories in Continual Learning, 2019. 1, 2

[10] Jianting Chen. Efficient core-set selection for deep learning through squared loss minimization. In *Proc. of ICML*, pages 1–10, 2025. 2

[11] Zhiyuan Chen and Bing Liu. *Lifelong Machine Learning*. Morgan & Claypool Publishers, 2018. 1

[12] Seungyoon Choi, Wonjoong Kim, Sungwon Kim, Yeonjun In, Sein Kim, and Chanyoung Park. DSLR: Diversity Enhancement and Structure Learning for Rehearsal-based Graph Continual Learning. In *Proceedings of the ACM on Web Conference 2024*, pages 733–744, 2024. 2

[13] Masih Eskandar, Tooba Imtiaz, Davin Hill, Zifeng Wang, and Jennifer G. Dy. STAR: stability-inducing weight perturbation for continual learning. In *Proc. of ICLR*, 2025. 6, 1

[14] Kamil Faber, Roberto Corizzo, Bartlomiej Sniezynski, and Nathalie Japkowicz. Lifelong continual learning for anomaly detection: New challenges, perspectives, and insights. *IEEE Access*, 12:41364–41380, 2024. 1

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proc. of CVPR*, pages 770–778, 2016. 6, 1

[16] Muyang He, Shuo Yang, Tiejun Huang, and Bo Zhao. Large-scale dataset pruning with dynamic uncertainty. In *Proc. of CVPR*, pages 7713–7722, 2024. 1, 2

[17] Stella Ho, Ming Liu, Lan Du, Longxiang Gao, and Yong Xiang. Prototype-Guided Memory Replay for Continual Learning. *IEEE Transactions on Neural Networks and Learning Systems*, 35:10973–10983, 2024. 2

[18] Wei Huang, Yunxiao Zhang, Shangmin Guo, Yu-Ming Shang, and Xiangling Fu. Dynimpt: A dynamic data selection method for improving model training efficiency. *IEEE Trans. Knowl. Data Eng.*, 37:239–252, 2025. 2

[19] Krishnateja Killamsetty, Durga S, Ganesh Ramakrishnan, Abir De, and Rishabh Iyer. Grad-match: Gradient matching based data subset selection for efficient deep model training. In *Proc. of ICML*, pages 5464–5474, 2021. 2

[20] D E Kirk. Optimal control theory: An introduction. *Positively Aware the Monthly Journal of the Test Positive Aware Network*, 23:13–5, 2004. 4

[21] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114:3521–3526, 2017. 1

[22] Marcus Klasson, Hedvig Kjellström, and Cheng Zhang. Learn the time to learn: Replay scheduling in continual learning. *Trans. Mach. Learn. Res.*, 2023, 2023. 2

[23] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. Master's thesis, University of Tront, 2009. 6

[24] Dhireesha Kudithipudi, Mario Aguilar-Simon, Jonathan Babb, Maxim Bazhenov, Douglas Blackiston, Josh Bongard, Andrew P Brna, Suraj Chakravarthi Raja, Nick Cheney, Jeff Clune, et al. Biological underpinnings for lifelong learning machines. *Nature Machine Intelligence*, 4:196–210, 2022. 1

[25] Ya Le and Xuan Yang. Tiny imagenet visual recognition challenge. *CS 231N*, 7:3, 2015. 6

[26] Sihao Liu, Yibo Yang, Xiaojie Li, David A. Clifton, and Bernard Ghanem. Enhancing online continual learning with plug-and-play state space model and class-conditional mixture of discretization. In *Proc. of CVPR*, pages 20502–20511, 2025. 2

[27] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Proc. of NeurIPS*, pages 6467–6476, 2017. 2, 6

[28] Adyasha Maharana, Prateek Yadav, and Mohit Bansal. D2 pruning: Message passing for balancing diversity & difficulty in data pruning. In *Proc. of ICLR*, 2024. 2

[29] Saumyaranjan Mohanty, Chimata Anudeep, and Konda Reddy Mopuri. Noise-free loss gradients: A surprisingly effective baseline for coreset selection. *Trans. Mach. Learn. Res.*, 4:1–19, 2025. 2

[30] Truong Thao Nguyen, Balazs Gerofi, Edgar Josafat Martinez-Noriega, François Trahay, and Mohamed Wahib. KAKURENBO: adaptively hiding samples in deep neural network training. In *Proc. of NeurIPS*, pages 1–23, 2023. 2

[31] Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. Deep learning on a data diet: Finding important examples early in training. In *Proc. of NeurIPS*, pages 20596–20607, 2021. 2

[32] Omead Pooladzandi, David Davini, and Baharan Mirzasoleiman. Adaptive second order coresets for data-efficient machine learning. In *Proc. of ICML*, pages 17848–17869, 2022. 2

[33] Ziheng Qin, Kai Wang, Zangwei Zheng, Jianyang Gu, Xiangyu Peng, Zhaopan Xu, Daquan Zhou, Lei Shang, Baigui Sun, Xuansong Xie, and Yang You. Infobatch: Lossless training speed up by unbiased dynamic data pruning. In *Proc. of ICLR*, 2024. 2

[34] Alessio Quercia, Abigail Morrison, Hanno Scharr, and Ira Assent. SGD biased towards early important samples for efficient training. In *Proc. of ICDM*, pages 1289–1294, 2023. 2

[35] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. iCaRL: Incremental classifier and representation learning. In *Proc. of CVPR*, 2017. 2, 6, 1

[36] Matthew Riemer, Ignacio Cases, Robert Ajemian, Miao Liu, Irina Rish, Yuhai Tu, and and Gerald Tesauro. Learning to Learn without Forgetting by Maximizing Transfer and Minimizing Interference. In *Proc. of ICLR*, 2018. 2

[37] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *CoRR*, abs/1606.04671, 2016. 1

[38] Fahad Sarfraz, Bahram Zonooz, and Elahe Arani. Beyond Unimodal Learning: The Importance of Integrating Multiple Modalities for Lifelong Learning, 2024. 1

[39] James Seale Smith, Lazar Valkov, Shaunak Halbe, Vyshnavi Gutta, Rogério Feris, Zsolt Kira, and Leonid Karlinsky. Adaptive memory replay for continual learning. In *Proc. of CVPR*, pages 3605–3615, 2024. 2

[40] Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari Morcos. Beyond neural scaling laws: beating power law scaling via data pruning. In *Proc. of NeurIPS*, pages 19523–19536, 2022. 1, 2

[41] Mariya Toneva, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J. Gordon. An empirical study of example forgetting during deep neural network learning. In *Proc. of ICLR*, 2019. 2

[42] Gido M. van de Ven, Tinne Tuytelaars, and Andreas S. Tolias. Three types of incremental learning. *Nature Machine Intelligence*, 4:1185–1197, 2022. 1

[43] Maorong Wang, Nicolas Michel, Ling Xiao, and Toshihiko Yamasaki. Improving plasticity in online continual learning via collaborative learning. In *Proc. of CVPR*, pages 23460–23469, 2024. 2

[44] Xin Wang, Yudong Chen, and Wenwu Zhu. A survey on curriculum learning. *IEEE Trans. Pattern Anal. Mach. Intell.*, 44:4555–4576, 2022. 2

[45] Hongwei Yan, Liyuan Wang, Kaisheng Ma, and Yi Zhong. Orchestrate latent expertise: Advancing online continual learning with multi-level supervision and reverse self-distillation. In *Proc. of CVPR*, pages 23670–23680, 2024. 2

[46] Shipeng Yan, Jiangwei Xie, and Xuming He. DER: dynamically expandable representation for class incremental learning. In *Proc. of CVPR*, pages 3014–3023, 2021. 2, 6

[47] Rui Yang, Shuang Wang, Huan Zhang, Siyuan Xu, YanHe Guo, Xiutiao Ye, Biao Hou, and Licheng Jiao. Knowledge Decomposition and Replay: A Novel Cross-modal Image-Text Retrieval Continual Learning Method. In *Proc. of ACM MM*, pages 6510–6519, 2023. 2

[48] Shuo Yang, Zeke Xie, Hanyu Peng, Min Xu, Mingming Sun, and Ping Li. Dataset pruning: Reducing training data by examining generalization influence. In *Proc. of ICLR*, 2023. 2

[49] Hyunsik Yoo, Seongku Kang, and Hanghang Tong. Continual recommender systems. In *Proc. of CIKM*, pages 6857–6860, 2025. 1

[50] Dianzhi Yu, Conghao Xiong, Yankai Chen, Wenqian Cui, Xinni Zhang, Yifei Zhang, Hao Chen, Joseph J. Y. Sung, and Irwin King. Consurv: Multimodal continual learning for survival analysis. In *Proc. of AAAI*, pages 27899–27907, 2026. 1

[51] Dianzhi Yu, Xinni Zhang, Yankai Chen, Aiwei Liu, Yifei Zhang, Philip S. Yu, and Irwin King. Recent advances of multimodal continual learning: A comprehensive survey. *IEEE Transactions on Neural Networks and Learning Systems*, pages 1–21, 2026. 1

[52] Yu-Chu Yu, Chi-Pin Huang, Jr-Jen Chen, Kai-Po Chang, Yung-Hsuan Lai, Fu-En Yang, and Yu-Chiang Frank Wang. Select and Distill: Selective Dual-Teacher Knowledge Transfer for Continual Learning on Vision-Language Models. In *Proc. of ECCV*, pages 219–236, 2025. 1

[53] Xi Zhang, Feifei Zhang, and Changsheng Xu. VQACL: A Novel Visual Question Answering Continual Learning Setting. In *Proc. of CVPR*, pages 19102–19112, 2023. 1

[54] Xinni Zhang, Yankai Chen, Chenhao Ma, Yixiang Fang, and Irwin King. Influential Exemplar Replay for Incremental Learning in Recommender Systems. In *Proc. of AAAI*, pages 9368–9376, 2024. 2

[55] Yifei Zhang, Hao Zhu, Alysa Ziying Tan, Dianzhi Yu, Longtao Huang, and Han Yu. pFedMxF: Personalized Federated Class-Incremental Learning with Mixture of Frequency Aggregation. In *Proc. of CVPR*, pages 30640–30650, 2025. 1

[56] Bowen Zheng, Da-Wei Zhou, Han-Jia Ye, and De-Chuan Zhan. Task-agnostic guided feature expansion for class-incremental learning. In *Proc. of CVPR*, pages 10099–10109, 2025. 2, 6

[57] Haizhong Zheng, Rui Liu, Fan Lai, and Atul Prakash. Coverage-centric coreset selection for high pruning rates. In *Proc. of ICLR*, 2023. 2

[58] Da-Wei Zhou, Qi-Wei Wang, Han-Jia Ye, and De-Chuan Zhan. A model or 603 exemplars: Towards memory-efficient class-incremental learning. In *Proc. of ICLR*, 2023. 6, 1

[59] Dawei Zhou, Zi-Wen Cai, Han-Jia Ye, Lijun Zhang, and De-Chuan Zhan. Dual consolidation for pre-trained model-based domain-incremental learning. In *Proc. of CVPR*, pages 20547–20557, 2025. 7

[60] Hao Zhu, Yifei Zhang, Junhao Dong, and Piotr Koniusz. BiLoRA: Almost-Orthogonal Parameter Spaces for Continual Learning. In *Proc. of CVPR*, pages 25613–25622, 2025. 1

Smart Replay: Adaptive Scheduling of Memory Rehearsal for Computational Resource-Aware Incremental Learning

Supplementary Material

A. More Experiment Details

Backbone architectures. We employ **ResNet-18** [15] as the backbone network on CIFAR-100. The input images have a resolution of $32 \times 32 \times 3$. The model starts with an embedding layer consisting of a 3×3 convolution with 64 channels, followed by an encoder composed of $[2, 2, 2, 2]$ BasicBlocks with channel sizes $[64, 128, 256, 512]$. The total number of parameters is 11.2M. We adopt **ResNet-34** [15] as the backbone network on Tiny-ImageNet. The input resolution is $224 \times 224 \times 3$. The embedding layer is a 7×7 convolution with 64 channels, followed by a max pooling layer. The encoder consists of $[3, 4, 6, 3]$ BasicBlocks with channel sizes $[64, 128, 256, 512]$. The total number of parameters is 21.8M. We use a two-layer **MLP** on Rotated MNIST. The input is flattened to a vector of $28 \times 28 = 784$ dimensions, followed by two hidden layers with 256 and 128 units, respectively, using ReLU activation. The model contains 235.1K parameters.

IL methods. Experience replay directly applies the replay strategy without modifying either the loss function or the model structure. Task samples and memory samples are treated equally and jointly used for gradient updates. **iCaRL** [35] introduces knowledge distillation. When learning the current task, the frozen model from the previous stage serves as the teacher. Each mini-batch is fed to both the current model and the frozen model to compute a distillation loss based on their outputs. The final loss is the sum of the task loss and the distillation loss. **MEMO** [58] is a lightweight, architecture-based method. Only the high-level modules are expanded when learning a new task, while the low-level layers are shared across tasks. For each new task, a parallel module is added and its outputs are concatenated with the previous ones. The models for previous tasks are frozen to preserve historical knowledge. In the ResNet-based models, we expand the last group of BasicBlocks. In the MLP models, all hidden layers are expanded. The shared lower layers remain trainable. Additionally, MEMO introduces an auxiliary classifier that distinguishes between new-task and previous-task features. The auxiliary loss is added to the task loss for model optimization. **STAR** [13] adopts a novel loss function that exploits the worst-case parameter perturbation that reduces the KL-divergence of model predictions with that of its local parameter neighborhood to promote stability and alleviate forgetting.

Training details. For CIFAR-100 and Tiny-ImageNet, we use random horizontal flipping and random cropping. We employ SGD with a momentum coefficient of 0.9. The weight decay is set to 1×10^{-4} for Tiny-ImageNet, and 5×10^{-4} for CIFAR-100 and Rotated-MNIST. The learning rate follows a cosine annealing schedule with a minimum value of 1×10^{-4}.

Baseline setting. For the limited-memory setting, we evaluate fixed replay ratios of $\lambda = 0.1$, 0.2, 0.3, and 0.4. Since the number of stored samples is relatively small in this setting, using an excessively large replay ratio often leads to poor performance: a large proportion of computational resources would be consumed by replaying a small set of samples, causing overfitting, while the new-task samples would suffer from insufficient training due to limited resources. For the unlimited-memory setting, we adopt fixed replay ratios of $\lambda = 0.2, 0.4, 0.6$, and 0.8, which are evenly distributed to explore the performance of IL models under different proportions of replay.

B. Last Forgetting Results

Tables 4 and 5 report the results of the last forgetting metric. Under the limited-memory setting, $\lambda \nearrow$ and Smart Replay show clear advantages in mitigating forgetting. By adjusting the replay ratio from low to high as training progresses, Smart Replay effectively alleviates the loss of historical knowledge and avoids overfitting to memory samples. Some forgetting scores are negative because, under a limited number of training epochs, the performance after replay training surpasses the accuracy achieved during the initial training. Within the baseline comparisons, a higher level of forgetting is observed when $\lambda = 0.1$, mainly because the replay resources are too limited to effectively counter forgetting. Conversely, at $\lambda = 0.4$, certain models also exhibit higher forgetting levels due to over-rehearsal: the model tends to overfit a small set of replayed samples, thereby losing generalization ability and yielding lower accuracy on the test data.

Under the unlimited-memory setting, the configuration with $\lambda = 0.8$ achieves the best performance in terms of the forgetting metric. However, its average accuracy is relatively lower. The reason lies in the fixed computational budget: a high replay ratio ($\lambda = 0.8$) dedicates a large portion of the resources to replaying past samples, leaving only limited capacity for learning the new task during its initial training phase. As a result, new tasks fail to reach full convergence and exhibit lower initial accuracy. In subsequent replay stages, since full data from previous tasks are available, these initially under-converged tasks can continue to improve during rehearsal, leading to higher accura-

Table 4. Last forgetting scores (%) under the limited-memory setting with different replay ratios (λ) and resource budgets (ϑ)

Replay	CIFAR-100			Tiny-ImageNet			Rotated-MNIST		
Schedule	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2
ER w/ limited memory									
Union	37.29	37.30	37.30	33.90	36.26	37.57	7.16	6.50	6.17
λ=0.1	47.12	41.28	38.23	54.61	48.51	42.14	15.36	13.00	11.40
λ=0.2	38.21	36.46	37.23	40.59	37.76	37.14	8.82	7.83	6.65
λ=0.3	37.21	38.35	37.65	36.31	36.51	37.97	6.25	5.91	5.64
λ=0.4	38.41	38.26	39.27	34.32	37.44	38.02	4.82	4.88	4.43
$\lambda \nearrow$	**29.44**	**31.88**	**33.63**	**17.17**	23.11	26.62	2.88	2.71	2.87
$\lambda \searrow$	44.26	44.93	43.91	43.52	45.31	45.52	5.77	5.34	5.19
Smart	30.75	33.72	36.51	31.54	**22.67**	**25.02**	**2.17**	**1.97**	**1.80**
iCaRL w/ limited memory									
Union	36.50	36.00	36.24	36.00	38.94	40.96	2.49	2.03	1.79
λ=0.1	42.00	39.17	38.18	49.34	43.79	39.16	7.25	5.72	4.87
λ=0.2	37.80	37.33	37.73	40.37	38.52	39.43	2.97	2.52	2.18
λ=0.3	36.93	36.58	36.53	38.99	39.04	40.77	1.70	1.22	1.05
λ=0.4	36.17	36.27	35.12	36.50	39.04	40.09	0.61	0.39	0.35
$\lambda \nearrow$	**25.88**	**26.55**	**27.34**	**20.94**	25.57	28.36	**-2.05**	**-2.14**	**-2.14**
$\lambda \searrow$	42.28	40.29	38.68	47.21	47.69	47.88	0.54	0.33	0.24
Smart	29.96	30.85	31.65	27.06	**23.59**	**26.88**	0.56	-0.11	-0.23
MEMO w/ limited memory									
Union	33.24	33.30	32.85	31.07	31.43	31.43	24.02	20.74	18.96
λ=0.1	40.85	36.35	35.10	51.66	45.43	40.68	43.22	37.99	36.29
λ=0.2	34.38	33.36	33.31	38.78	36.10	35.30	31.43	27.05	23.70
λ=0.3	33.23	32.90	33.57	33.30	31.88	31.50	22.22	18.40	16.87
λ=0.4	34.21	35.31	35.23	31.40	30.06	32.69	16.23	15.94	14.97
$\lambda \nearrow$	29.33	32.21	33.15	**14.76**	**19.52**	23.87	**6.24**	5.85	5.39
$\lambda \searrow$	40.61	40.55	39.32	41.77	41.38	41.03	18.32	18.94	18.07
Smart	**28.28**	**23.26**	**20.15**	28.64	20.48	**20.29**	8.84	**3.24**	**0.34**
STAR w/ limited memory									
Union	34.15	33.33	30.94	32.19	33.48	33.91	7.18	6.60	6.31
λ=0.1	50.55	44.16	38.48	56.23	52.60	47.86	15.47	13.14	11.51
λ=0.2	35.95	32.25	30.79	44.51	39.33	35.94	8.88	7.88	6.74
λ=0.3	33.74	31.65	31.34	36.54	33.90	33.69	6.26	6.08	5.80
λ=0.4	32.60	31.76	31.77	33.02	34.18	34.63	4.87	4.93	4.56
$\lambda \nearrow$	**27.08**	28.08	29.12	**14.33**	**17.02**	**21.49**	3.00	2.90	2.99
$\lambda \searrow$	38.13	36.45	34.54	43.20	44.29	42.67	5.82	5.43	5.34
Smart	32.09	**27.95**	**28.10**	25.88	21.62	21.54	**2.17**	**2.08**	**1.88**

Table 5. Last forgetting scores (%) under the unlimited-memory setting with different replay ratios (λ) and resource budgets (ϑ)

Replay	CIFAR-100			Tiny-ImageNet			Rotated-MNIST		
Schedule	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2
ER w/ unlimited memory									
Union	3.44	2.63	2.61	**-3.18**	**-4.92**	-4.66	-0.84	-1.14	-1.15
λ=0.2	28.30	23.85	20.23	40.74	34.40	27.56	7.35	5.97	5.34
λ=0.4	16.45	13.25	11.45	25.58	19.79	15.88	3.21	2.24	1.80
λ=0.6	8.50	6.56	5.68	14.00	10.59	7.32	0.91	0.17	0.12
λ=0.8	**-2.71**	**-2.47**	**-0.65**	-1.99	-3.73	**-4.69**	**-2.62**	**-2.76**	**-2.23**
$\lambda \nearrow$	2.04	0.83	1.04	7.66	4.61	2.54	0.96	0.45	**0.40**
$\lambda \searrow$	24.77	21.19	17.35	31.66	26.71	20.90	3.47	2.87	2.35
Smart	7.10	2.40	4.53	10.98	3.75	2.35	2.05	0.50	-0.50
iCaRL w/ unlimited memory									
Union	3.13	2.40	1.85	**-3.58**	**-5.16**	-4.23	-4.61	-4.40	-4.45
λ=0.2	26.48	21.66	18.23	37.77	30.92	24.43	2.52	1.63	1.25
λ=0.4	15.18	12.95	9.91	24.26	19.13	14.74	-0.78	-1.13	-1.24
λ=0.6	7.39	5.97	4.95	13.17	10.32	7.96	-3.18	-3.39	-3.44
λ=0.8	**-3.70**	**-3.40**	**-3.41**	-0.79	-3.60	**-5.28**	**-7.90**	**-7.41**	**-7.08**
$\lambda \nearrow$	0.13	-0.06	-1.54	5.74	2.08	1.33	-3.81	-3.54	-3.53
$\lambda \searrow$	26.64	22.07	18.16	34.19	28.36	24.40	-1.30	-1.39	-1.64
Smart	5.68	3.43	1.85	10.26	3.00	0.54	-1.19	-1.38	-1.23
MEMO w/ unlimited memory									
Union	3.93	3.88	4.09	**-0.23**	**-0.98**	**-1.61**	0.61	0.90	0.64
λ=0.2	25.61	21.15	16.87	38.91	34.21	27.98	30.00	26.68	23.41
λ=0.4	14.98	12.76	5.80	25.73	21.04	17.26	15.34	15.16	12.49
λ=0.6	8.45	7.03	6.13	15.86	11.53	9.72	6.78	6.56	5.17
λ=0.8	**-1.03**	**-1.03**	**-1.01**	1.61	0.57	0.20	**-2.91**	**-3.12**	**-2.62**
$\lambda \nearrow$	3.46	3.10	2.47	6.94	4.91	3.20	6.07	5.92	4.68
$\lambda \searrow$	24.89	20.05	16.75	36.21	29.64	25.46	19.89	19.58	17.97
Smart	6.15	5.80	4.80	12.17	4.80	0.46	13.58	13.99	11.31
STAR w/ unlimited memory									
Union	3.46	2.44	2.70	**-3.57**	**-4.12**	**-5.74**	-0.95	-1.21	-1.52
λ=0.2	30.35	25.11	20.86	46.70	39.01	32.87	6.59	5.25	4.73
λ=0.4	16.71	12.29	10.44	27.78	21.54	16.01	2.59	2.02	1.92
λ=0.6	8.32	6.05	5.05	13.87	10.14	7.71	0.39	-0.14	-0.30
λ=0.8	**-3.55**	**-3.38**	**-4.11**	-2.11	-4.02	-4.52	**-2.94**	**-2.91**	**-2.77**
$\lambda \nearrow$	2.61	1.16	-0.69	9.71	5.35	2.41	0.55	0.31	0.25
$\lambda \searrow$	24.76	19.45	17.23	32.68	27.51	21.66	2.56	2.12	1.79
Smart	4.71	3.21	2.08	11.61	3.07	0.07	1.78	1.30	1.05

cies than before and consequently producing negative forgetting values in the metric. In the same setting, Smart Replay achieves the second-best performance on CIFAR-100 and Tiny-ImageNet, while maintaining a high average accuracy. This result indicates that Smart Replay effectively balances the plasticity required for learning new tasks and the stability needed to retain previous knowledge.

C. Training Time Analysis

Table 6 reports the training time costs under the limited-memory setting across different datasets and computational budget configurations. The time costs under the unlimited-memory setting are similar, as the number of gradient descent steps and the algorithms remain unchanged. Additionally, the variations across different replay ratios are negligible. This observation indicates that the training time mainly depends on the learning algorithm and the computational resource parameter ϑ.

Both iCaRL and MEMO introduce additional computational overhead compared to the simple replay strategy, due to the cost of knowledge distillation in iCaRL and the structural expansion in MEMO. Although the additional module introduced by MEMO for each new task is relatively small in size, the cumulative time overhead becomes non-negligible as the number of tasks increases. ICaRL only employs the model from the previous stage as a teacher for knowledge distillation, and thus its additional computational cost does not scale with the number of tasks.

The parameter ϑ controls the computational budget: when $\vartheta > 0$, it allows the available computational resources to increase during the IL process, leading to longer overall training times.

D. More Training Epochs

In the main experiments, we set the computational budget baseline to 20 epochs (e=20) to emulate a resource-constrained scenario. To further verify the effect of increased computational resources, we extend the training budget to 60 and 100 epochs for CIFAR-100 and Tiny-ImageNet, respectively, and evaluate the average accuracy and last forgetting under different memory settings.

Under the limited-memory setting, the average accuracies are reported in Table 7. Smart Replay achieves consistently superior performance on Tiny-ImageNet. The configuration with $\lambda = 0.1$ delivers the second-best result, suggesting that when computation is sufficient but the num-

Table 6. Training time cost (s) under the limited-memory setting with different replay ratios (λ) and resource budgets (ϑ)

IL Method	Replay Ratio	CIFAR-100			Tiny-ImageNet			Rotated-MNIST		
		$\vartheta=0$	$\vartheta=0.1$	$\vartheta=0.2$	$\vartheta=0$	$\vartheta=0.1$	$\vartheta=0.2$	$\vartheta=0$	$\vartheta=0.1$	$\vartheta=0.2$
Base	Union	343	423	469	3164	4632	6032	103	119	157
	λ=0.1	338	404	472	3160	4621	6013	100	120	162
	λ=0.2	343	409	471	3151	4610	6023	104	119	163
	λ=0.3	335	406	469	3167	4615	6019	97	122	158
	λ=0.4	338	404	475	3145	4673	6043	102	116	159
	$\lambda\nearrow$	340	412	468	3139	4732	6050	101	115	163
	$\lambda\searrow$	343	419	477	3149	4729	6039	100	120	169
	Smart	342	405	471	3170	4640	6032	103	124	161
iCaRL	Union	395	479	550	4032	5891	7632	116	142	169
	λ=0.1	389	473	554	4010	5880	7612	120	140	175
	λ=0.2	389	473	550	3993	5901	7656	116	140	175
	λ=0.3	384	473	554	4044	5842	7630	121	136	180
	λ=0.4	393	477	552	4061	5860	7644	119	137	178
	$\lambda\nearrow$	385	471	557	4034	5849	7620	124	137	185
	$\lambda\searrow$	391	465	557	4059	5863	7648	120	139	180
	Smart	394	471	549	4058	5983	7632	113	143	178
MEMO	Union	401	489	571	4327	6313	8640	111	143	180
	λ=0.1	400	486	577	4330	6334	8630	110	142	180
	λ=0.2	401	484	577	4351	6321	8640	109	141	181
	λ=0.3	397	485	576	4338	6390	8638	110	142	182
	λ=0.4	399	485	573	4353	6370	8653	110	144	178
	$\lambda\nearrow$	400	496	579	4358	6381	8641	111	141	181
	$\lambda\searrow$	399	482	570	4350	6374	8659	110	144	183
	Smart	402	486	581	4293	6350	8641	108	138	185
STAR	Union	395	479	564	4301	6299	8592	106	137	168
	λ=0.1	400	470	565	4291	6301	8589	107	145	168
	λ=0.2	402	469	570	4301	6321	8600	109	141	170
	λ=0.3	397	477	566	4318	6302	8611	110	142	171
	λ=0.4	397	479	573	4313	6340	8623	110	142	173
	$\lambda\nearrow$	389	469	576	4316	6321	8611	109	142	171
	$\lambda\searrow$	399	472	571	4320	6319	8630	110	144	172
	Smart	401	480	567	4293	6324	8612	108	148	175

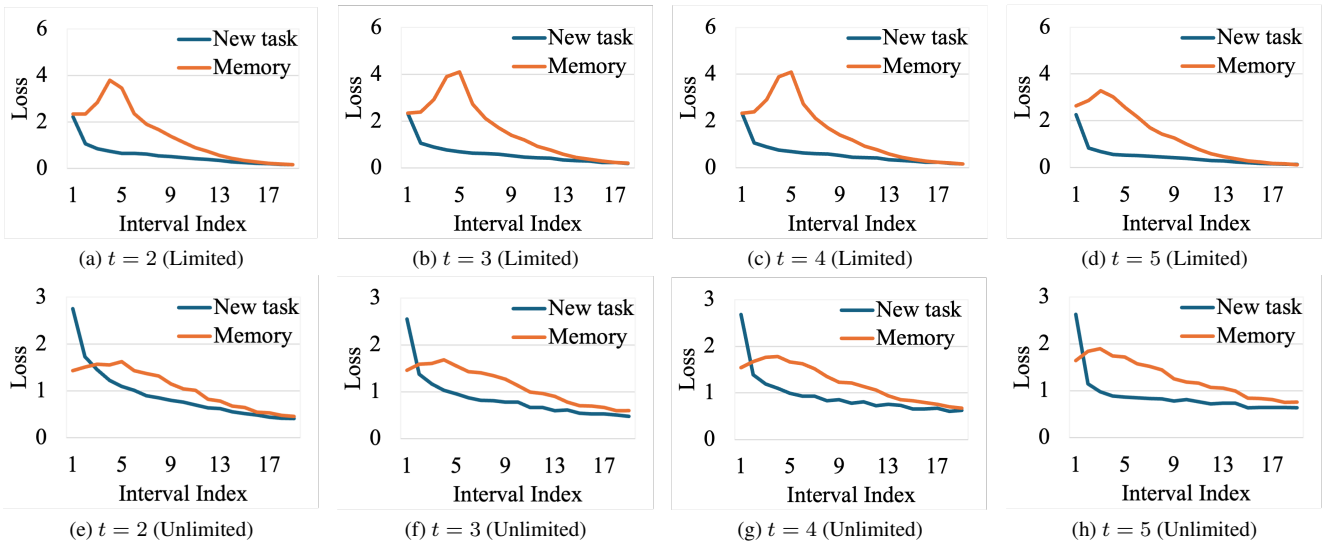
ber of memory samples is limited, a small replay ratio is sufficient to retain historical knowledge, while fixed and frequent replay may lead to degraded performance. On CIFAR-100, the results of the Base model are consistent with those on Tiny-ImageNet. iCaRL, due to its use of knowledge distillation, effectively mitigates the overfitting caused by frequent replay and achieves competitive performance even at larger λ values. However, under the MEMO model, the advantage of Smart Replay diminishes. We attribute this to inaccurate gradient norm estimation caused by parameter freezing, which limits the adaptation capability of Smart Replay. Furthermore, the last forgetting results in Table 9 show that Smart Replay achieves the smallest forgetting in most cases.

Under the unlimited memory setting, Table 8 demonstrates that Smart Replay achieves consistently superior performance on Tiny-ImageNet. The second-best performance is obtained with $\lambda = 0.6$, indicating that a moderate replay ratio performs better than extreme values when data availability is unconstrained. For CIFAR-100, which is relatively simpler, the abundance of computational resources and unlimited data availability allow both new and old tasks to fit effectively across different replay ratios. As a result, the advantage of Smart Replay is less pronounced. Further examination of the forgetting metric in Table 10 shows that Smart Replay and $\lambda = 0.8$ have an advantage, indicating that Smart Replay effectively mitigates forgetting. Overall, even under unconstrained data conditions, Smart Replay holds significant value in scenarios with limited computational resources and challenging tasks.

E. Learning Dynamic under Smart Replay

In Section 5.3, we present the evolution of the replay ratio under the influence of Smart Replay. Here, taking CIFAR-100 as an example, we illustrate in Figure 7 the dynamic loss curves of the new-task samples and the memory samples within each learning stage t. The overall patterns are consistent with the schematic shown in Figure 2: the new-task loss decreases monotonically, while the memory loss first increases and then gradually decreases.

Under the limited-memory setting, we set the parameters $\Delta\rho = 2$ and $\tau = 0.2$. At the beginning of training, most computational resources are allocated to the new task. Consequently, the new-task loss rapidly decreases, while the memory loss significantly rises. In the later phases, the



(a) $t = 2$ (Limited) (b) $t = 3$ (Limited) (c) $t = 4$ (Limited) (d) $t = 5$ (Limited)

(e) $t = 2$ (Unlimited) (f) $t = 3$ (Unlimited) (g) $t = 4$ (Unlimited) (h) $t = 5$ (Unlimited)

Figure 7. Learning dynamics in each stage with adaptive scheduling on CIFAR-100 under two memory settings

Table 7. Average accuracy (%) in the limited-memory setting with more training epochs

IL Method	Replay Ratio	CIFAR-100 e=60	CIFAR-100 e=100	Tiny-ImageNet e=60	Tiny-ImageNet e=100
Base	λ=0.1	64.91	64.97	47.19	47.97
	λ=0.2	64.11	64.42	46.32	46.17
	λ=0.3	64.34	64.57	45.19	44.82
	λ=0.4	64.10	64.73	43.95	43.74
	Smart	**65.79**	**67.41**	**50.04**	**48.26**
iCaRL	λ=0.1	65.88	67.00	45.93	46.55
	λ=0.2	66.57	67.51	45.76	45.29
	λ=0.3	66.23	67.61	45.32	44.79
	λ=0.4	66.49	68.29	45.21	45.35
	Smart	**67.44**	**70.48**	**46.92**	**48.59**
MEMO	λ=0.1	**66.78**	**67.35**	46.10	46.55
	λ=0.2	66.46	66.64	45.32	45.29
	λ=0.3	66.19	66.35	44.23	44.79
	λ=0.4	65.75	66.00	44.83	45.35
	Smart	66.42	66.19	**47.85**	**48.59**

Table 8. Average accuracy (%) in the unlimited-memory setting with more training epochs

IL Method	Replay Ratio	CIFAR-100 e=60	CIFAR-100 e=100	Tiny-ImageNet e=60	Tiny-ImageNet e=100
Base	λ=0.2	77.82	79.86	59.87	64.15
	λ=0.4	79.11	80.67	63.63	65.82
	λ=0.6	**79.57**	**80.77**	63.99	65.38
	λ=0.8	79.01	80.30	63.60	64.82
	Smart	79.42	80.75	**66.28**	**67.07**
iCaRL	λ=0.2	76.86	**79.86**	59.30	62.40
	λ=0.4	78.00	79.49	64.32	63.30
	λ=0.6	78.15	79.62	64.59	63.84
	λ=0.8	77.95	79.44	64.01	63.66
	Smart	**78.35**	79.47	**65.83**	**64.57**
MEMO	λ=0.2	77.15	78.32	60.23	63.64
	λ=0.4	77.99	78.77	62.40	64.37
	λ=0.6	78.03	**79.21**	63.29	64.79
	λ=0.8	**78.13**	78.86	63.19	64.57
	Smart	77.68	77.99	**64.30**	**65.37**

convergence gap between new and memory samples is captured by the Q-function, which progressively increases the replay ratio. This allocation shift provides more computational resources for memory data, allowing their losses to converge until both new-task and memory losses become aligned, thereby maintaining a balanced learning state between plasticity and stability.

Under the unlimited-memory setting, we set $\Delta\rho = 0.5$ and $\tau = 0.5$, ensuring a smaller resource skew during the early training stages, with the replay ratio controlled around 0.5. In the early phases, the rate of decline for new task losses and the rate of increase for memory losses both slow down. This is because the abundance of memory samples requires more resources to maintain stability. In the middle and later phases, the replay ratio is adaptively adjusted based on differences in convergence states, achieving a balanced minimization of both losses.

This demonstrates that the settings of $\Delta\rho$ and τ have a direct impact on the loss dynamics during early training. Depending on task difficulty and memory sample size, resources can be manually distributed in the early stages to either enhance plasticity or pursue stability. In the later phases, the Q-function adaptively balances the two losses and minimizes them.

F. Derivation of the Q-Function

In this section, we provide the detailed derivation of the Q-function approximation presented in Eq. (16). We start from the definition of the Q-function and progressively in-

Table 9. Last forgetting (%) in the limited-memory setting with more training epochs

IL Method	Replay Ratio	CIFAR-100 e=60	CIFAR-100 e=100	Tiny-ImageNet e=60	Tiny-ImageNet e=100
Base	λ=0.1	38.84	40.64	40.93	**37.49**
	λ=0.2	41.11	42.29	40.76	41.38
	λ=0.3	42.11	42.99	41.73	43.09
	λ=0.4	41.97	43.12	42.91	44.10
	Smart	**37.09**	**39.74**	**34.00**	38.36
iCaRL	λ=0.1	39.06	38.68	43.32	42.49
	λ=0.2	38.94	37.76	45.34	44.68
	λ=0.3	37.83	36.94	46.92	45.42
	λ=0.4	36.63	36.21	47.43	45.04
	Smart	**34.72**	**27.10**	**39.43**	**36.80**
MEMO	λ=0.1	34.54	36.51	31.23	30.56
	λ=0.2	36.69	36.89	31.42	30.64
	λ=0.3	36.24	37.07	31.95	32.43
	λ=0.4	36.34	37.89	32.63	33.00
	Smart	**26.82**	**24.90**	**29.03**	**27.50**

Table 10. Last forgetting (%) in the unlimited-memory setting with more training epochs

IL Method	Replay Ratio	CIFAR-100 e=60	CIFAR-100 e=100	Tiny-ImageNet e=60	Tiny-ImageNet e=100
Base	λ=0.2	9.35	14.19	23.72	16.47
	λ=0.4	6.30	8.49	14.59	10.33
	λ=0.6	6.17	6.35	11.48	8.26
	λ=0.8	**5.65**	**4.44**	3.86	6.56
	Smart	5.71	5.44	**3.31**	**2.71**
iCaRL	λ=0.2	9.05	12.67	22.12	12.41
	λ=0.4	6.21	7.69	15.33	8.76
	λ=0.6	4.76	5.00	12.14	8.00
	λ=0.8	**4.49**	**2.96**	5.32	6.32
	Smart	4.95	5.65	**4.92**	**2.48**
MEMO	λ=0.2	9.07	7.83	19.32	11.29
	λ=0.4	6.69	6.22	18.32	8.52
	λ=0.6	5.73	5.44	16.94	6.76
	λ=0.8	4.76	4.40	16.34	5.83
	Smart	**-0.92**	**3.53**	**14.93**	**0.06**

corporate the first-order Taylor expansion of the loss dynamics.

Step 1: Definition. Recall the definition of the Q-function:

$$Q(l_r^M, l_r^D, r; \lambda_r) = R(l_r^M, l_r^D; \lambda_r) + V(l_{r+1}^M, l_{r+1}^D, r+1). \tag{21}$$

Substituting Eqs. (9)-(10) yields

$$\begin{aligned} Q(l_r^M, l_r^D, r; \lambda_r) &= \Delta l_r^M + \Delta l_r^D - \epsilon(\lambda_r - \lambda_{r-1})^2 \\ &+ (l_{r+1}^M - l_{r+1}^M e^{\rho_M \frac{r+1-R}{R}}) + (l_{r+1}^D - l_{r+1}^D e^{\rho_D \frac{r+1-R}{R}}). \end{aligned} \tag{22}$$

Step 2: Simplification. Let $w_r^M = e^{\rho_M (r-R)/R}$ and $w_r^D = e^{\rho_D (r-R)/R}$. Using $l_{r+1}^M = l_r^M - \Delta l_r^M$ and $l_{r+1}^D = l_r^D - \Delta l_r^D$, we obtain

$$\begin{aligned} Q(l_r^M, l_r^D, r; \lambda_r) &= w_{r+1}^M \Delta l_r^M + w_{r+1}^D \Delta l_r^D - \epsilon(\lambda_r - \lambda_{r-1})^2 \\ &+ l_r^M (1 - w_{r+1}^M) + l_r^D (1 - w_{r+1}^D). \end{aligned} \tag{23}$$

Since the last two terms are independent of λ_r, they can be discarded. Approximating $w_{r+1}^M \approx w_r^M$ and $w_{r+1}^D \approx w_r^D$ gives

$$Q(l_r^M, l_r^D, r; \lambda_r) = w_r^M \Delta l_r^M + w_r^D \Delta l_r^D - \epsilon(\lambda_r - \lambda_{r-1})^2, \tag{24}$$

where w_r^M and w_r^D are time-adaptive exponential weights.

Step 3: Linearization of loss dynamics. We linearize the loss changes via the first-order Taylor expansion in Eqs. (13)–(14):

$$\begin{aligned} \Delta l_r^M &\approx \eta \big[\lambda_r \|\nabla L_r^M\|^2 + (1-\lambda_r) \nabla L_r^M \cdot \nabla L_r^D\big], \\ \Delta l_r^D &\approx \eta \big[\lambda_r \nabla L_r^D \cdot \nabla L_r^M + (1-\lambda_r) \|\nabla L_r^D\|^2\big]. \end{aligned} \tag{25}$$

Step 4: Substitution into $Q(\cdot)$. Substituting the above expansions into Eq. (24) yields

$$\begin{aligned} Q(l_r^M, l_r^D, r; \lambda_r) &= \eta w_r^M \big[\lambda_r \|\nabla L_r^M\|^2 + (1-\lambda_r) \nabla L_r^M \cdot \nabla L_r^D\big] \\ &+ \eta w_r^D \big[\lambda_r \nabla L_r^D \cdot \nabla L_r^M + (1-\lambda_r) \|\nabla L_r^D\|^2\big] \\ &- \epsilon(\lambda_r - \lambda_{r-1})^2. \end{aligned} \tag{26}$$

Step 5: Collecting λ_r terms. After expanding and merging like terms, we obtain

$$\begin{aligned} Q(l_r^M, l_r^D, r; \lambda_r) &= \eta \lambda_r [w_r^M \|\nabla L_r^M\|^2 - w_r^D \|\nabla L_r^D\|^2 \\ &+ (w_r^D - w_r^M) \nabla L_r^M \cdot \nabla L_r^D] \\ &- \epsilon(\lambda_r - \lambda_{r-1})^2. \end{aligned} \tag{27}$$

Step 6: Reformulation. Let β denote the angle between the two task gradients, $\nabla L_r^M \cdot \nabla L_r^D = \|\nabla L_r^M\| \|\nabla L_r^D\| \cos\beta$, and define $\gamma_r = w_r^D \|\nabla L_r^D\|^2$. Then Eq. (27) can be rewritten as

$$\begin{aligned} &Q(l_r^M, l_r^D, r; \lambda_r) \\ &= \eta \gamma_r \lambda_r \left[\frac{w_r^M}{w_r^D} \frac{\|\nabla L_r^M\|^2}{\|\nabla L_r^D\|^2} + \left(1 - \frac{w_r^M}{w_r^D}\right) \frac{\|\nabla L_r^M\|}{\|\nabla L_r^D\|} \cos\beta - 1 \right] \\ &- \epsilon(\lambda_r - \lambda_{r-1})^2, \end{aligned} \tag{28}$$

which corresponds to Eq. (16) in the main text.

G. Standard Deviation of Comparison Experiments

Tables 11 and 12 present the standard deviations of the comparison experiment results obtained from multiple repetitions.

Table 11. The standard deviation of average accuracies (%) under the limited-memory setting

Replay	CIFAR-100			Tiny-ImageNet			Rotated-MNIST		
Schedule	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2
ER w/ limited memory									
Union	0.23	0.31	0.15	0.23	0.33	0.31	0.05	0.06	0.09
$\lambda = 0.1$	0.26	0.27	0.32	0.14	0.14	0.10	0.05	0.04	0.01
$\lambda = 0.2$	0.32	0.26	0.21	0.18	0.27	0.16	0.04	0.01	0.06
$\lambda = 0.3$	0.22	0.22	0.15	0.27	0.25	0.25	0.08	0.06	0.03
$\lambda = 0.4$	0.22	0.20	0.17	0.28	0.20	0.18	0.08	0.02	0.04
$\lambda \nearrow$	0.31	0.33	0.30	0.21	0.17	0.12	0.01	0.04	0.07
$\lambda \searrow$	0.28	0.18	0.18	0.09	0.35	0.08	0.01	0.02	0.08
Smart	0.22	0.24	0.27	0.21	0.16	0.10	0.09	0.04	0.06
iCaRL w/ limited memory									
Union	0.28	0.34	0.29	0.33	0.23	0.09	0.08	0.08	0.01
$\lambda = 0.1$	0.26	0.25	0.33	0.21	0.34	0.27	0.02	0.04	0.08
$\lambda = 0.2$	0.30	0.25	0.29	0.14	0.29	0.34	0.03	0.07	0.01
$\lambda = 0.3$	0.25	0.16	0.32	0.33	0.08	0.23	0.05	0.08	0.03
$\lambda = 0.4$	0.33	0.29	0.24	0.31	0.26	0.16	0.00	0.07	0.09
$\lambda \nearrow$	0.33	0.35	0.16	0.23	0.22	0.18	0.00	0.06	0.03
$\lambda \searrow$	0.16	0.19	0.16	0.14	0.20	0.21	0.01	0.07	0.05
Smart	0.21	0.33	0.31	0.18	0.24	0.31	0.06	0.01	0.05
MEMO w/ limited memory									
Union	0.34	0.19	0.31	0.16	0.22	0.32	0.02	0.04	0.06
$\lambda = 0.1$	0.33	0.34	0.31	0.11	0.15	0.07	0.07	0.09	0.01
$\lambda = 0.2$	0.29	0.19	0.33	0.25	0.30	0.07	0.04	0.06	0.00
$\lambda = 0.3$	0.18	0.28	0.22	0.25	0.07	0.13	0.03	0.10	0.06
$\lambda = 0.4$	0.20	0.23	0.34	0.19	0.11	0.12	0.04	0.04	0.05
$\lambda \nearrow$	0.28	0.28	0.30	0.14	0.23	0.23	0.01	0.04	0.02
$\lambda \searrow$	0.28	0.29	0.25	0.17	0.07	0.33	0.08	0.07	0.09
Smart	0.24	0.17	0.27	0.13	0.22	0.22	0.05	0.01	0.00
STAR w/ limited memory setting									
Union	0.24	0.28	0.21	0.11	0.07	0.15	0.04	0.04	0.01
$\lambda = 0.1$	0.22	0.32	0.22	0.19	0.30	0.13	0.09	0.09	0.08
$\lambda = 0.2$	0.33	0.31	0.17	0.33	0.13	0.27	0.04	0.08	0.00
$\lambda = 0.3$	0.17	0.16	0.30	0.25	0.11	0.13	0.01	0.03	0.07
$\lambda = 0.4$	0.21	0.16	0.29	0.21	0.18	0.15	0.10	0.04	0.08
$\lambda \nearrow$	0.18	0.23	0.32	0.16	0.16	0.30	0.05	0.02	0.01
$\lambda \searrow$	0.34	0.18	0.22	0.18	0.19	0.22	0.02	0.05	0.07
Smart	0.31	0.20	0.25	0.25	0.29	0.31	0.05	0.06	0.00

Table 12. The standard deviation of average accuracies (%) under the unlimited-memory setting

Replay	CIFAR-100			Tiny-ImageNet			Rotated-MNIST		
Schedule	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2	ϑ=0	ϑ=0.1	ϑ=0.2
ER w/ unlimited memory									
Union	0.23	0.25	0.19	0.36	0.41	0.41	0.08	0.01	0.03
$\lambda = 0.1$	0.28	0.16	0.28	0.34	0.45	0.41	0.04	0.03	0.07
$\lambda = 0.2$	0.19	0.24	0.18	0.40	0.37	0.46	0.04	0.09	0.05
$\lambda = 0.3$	0.22	0.23	0.23	0.28	0.29	0.39	0.08	0.10	0.01
$\lambda = 0.4$	0.11	0.30	0.12	0.41	0.46	0.27	0.04	0.03	0.08
$\lambda \nearrow$	0.29	0.18	0.23	0.52	0.26	0.32	0.02	0.01	0.05
$\lambda \searrow$	0.13	0.15	0.30	0.41	0.40	0.30	0.04	0.09	0.04
Smart	0.14	0.30	0.15	0.34	0.35	0.42	0.09	0.06	0.00
iCaRL w/ unlimited memory									
Union	0.14	0.23	0.28	0.44	0.54	0.38	0.04	0.03	0.09
$\lambda = 0.1$	0.13	0.26	0.21	0.34	0.43	0.33	0.07	0.06	0.04
$\lambda = 0.2$	0.17	0.12	0.16	0.27	0.40	0.43	0.03	0.09	0.09
$\lambda = 0.3$	0.27	0.25	0.20	0.33	0.32	0.39	0.07	0.10	0.06
$\lambda = 0.4$	0.19	0.28	0.30	0.52	0.36	0.33	0.07	0.04	0.09
$\lambda \nearrow$	0.19	0.25	0.17	0.29	0.37	0.37	0.10	0.00	0.03
$\lambda \searrow$	0.19	0.18	0.14	0.36	0.36	0.48	0.05	0.02	0.01
Smart	0.12	0.29	0.29	0.55	0.26	0.27	0.01	0.01	0.03
MEMO w/ unlimited memory									
Union	0.18	0.26	0.27	0.54	0.44	0.55	0.02	0.07	0.03
$\lambda = 0.1$	0.12	0.29	0.27	0.47	0.27	0.33	0.01	0.10	0.06
$\lambda = 0.2$	0.11	0.24	0.18	0.31	0.29	0.36	0.02	0.01	0.04
$\lambda = 0.3$	0.21	0.16	0.20	0.50	0.51	0.43	0.02	0.05	0.02
$\lambda = 0.4$	0.11	0.25	0.22	0.55	0.49	0.39	0.01	0.05	0.05
$\lambda \nearrow$	0.11	0.12	0.16	0.30	0.35	0.35	0.08	0.02	0.05
$\lambda \searrow$	0.25	0.15	0.13	0.42	0.50	0.43	0.01	0.06	0.03
Smart	0.23	0.19	0.16	0.47	0.54	0.30	0.07	0.08	0.00
STAR w/ unlimited memory									
Union	0.29	0.10	0.19	0.28	0.34	0.35	0.08	0.05	0.04
$\lambda = 0.1$	0.21	0.25	0.25	0.35	0.46	0.35	0.10	0.09	0.08
$\lambda = 0.2$	0.19	0.12	0.28	0.45	0.28	0.38	0.10	0.01	0.03
$\lambda = 0.3$	0.29	0.25	0.19	0.48	0.29	0.27	0.05	0.03	0.04
$\lambda = 0.4$	0.14	0.23	0.11	0.28	0.33	0.38	0.05	0.03	0.05
$\lambda \nearrow$	0.15	0.29	0.23	0.41	0.37	0.45	0.06	0.05	0.08
$\lambda \searrow$	0.12	0.29	0.26	0.54	0.48	0.50	0.04	0.01	0.01
Smart	0.31	0.20	0.25	0.25	0.29	0.31	0.05	0.06	0.00